The Cold Start Problem in Serverless Computing

Michael Murphy
Villanova University, Computer Systems I
4/18/26

1. Introduction

Cloud computing has evolved through a steady shift toward smaller, more precise ways of handling hardware. This journey moved from renting entire physical servers to using Virtual Machines (VMs), then to Containers, and finally to Function-As-A-Service (FaaS), or commonly called serverless computing. Under this model, developers only need to worry about their specific code, while the cloud provider manages everything from physical hardware to the software environment. Because this system can “scale to zero”, it is incredibly cost-effective and simple to run, as resources are only used and paid for the moment an event triggers them.

This introduces a major systems-level hurdle called the “cold start”. This is essentially a "performance tax” or delay that occurs when a request arrives, but there is no active environment ready to handle it. In these moments, the provider must perform a high-speed series of background tasks: they have to grab space on a physical server, set up a Micro-Virtual Machine, download the code and its libraries, and boot up the language environment (such as Java Virtual Machine). This whole process can take between a few hundred milliseconds to several seconds. For time-sensitive tasks like web apps or real-time data processing, this delay can be a significant problem for the user experience.

Historically, cold starts were viewed primarily as a performance bottleneck affecting user experience. However, the technical landscape shifted significantly in August 2025 when major providers like AWS began billing for the INIT phase of function execution. This change turned an engineering challenge into a direct financial liability, making the optimization of startup sequences a critical requirement for production-grade systems.

This report explores the technical origins of cold start latency within the modern system stack. I will analyze the trade-offs between hardware-level isolation and startup speed, specifically examining the role of AWS Firecracker and lightweight virtualization. I will also evaluate modern mitigation strategies, such as checkpoint/restore mechanisms and snapshot-based restoration, to determine how near-instant compute can be achieved without compromising the security boundaries essential for multi-tenant cloud environments.

2. The Anatomy of the Cold Start

To understand the Cold Start, we must look past the high-level API and examine the interaction between the cloud provider's software and the underlying physical hardware. This process is bifurcated into two major domains: the Control Plane and the Data Plane.

2.1 The Latency of the Control Plane

You have to look at the Control Plane as the brain of the whole serverless setup, and when a request hits, it triggers a massive, high-speed matchmaking process. First, the system has to run its placement logic, which is basically a giant query to a global state store to find a node that has enough CPU and RAM available. It uses these complex bin-packing algorithms to try and cram as many functions as possible into the hardware without letting a single function hog all the resources. If there isn't a "warm" instance already waiting, the system is forced onto the "Cold Path". This is where things slow down significantly because it has to go and fetch your deployment package (usually a zip file) from a storage bucket. If your code is bloated, or the packet is physically far from the node, you're already losing hundreds of milliseconds waiting on the download. Then comes the "network plumbing", which was once the biggest bottleneck of them all. The system has to dynamically set up a VPC interface, create an Elastic Network Interface, and update the host's routing table just so the function can communicate with the internet. WHile AWS improved this with remote NAT mapping, it's still a lot of heavy lifting to do in the split second before any code is run.

2.2 Virtualization Overhead

In traditional cloud computing, hardware is partitioned using Virtual Machines (VM). While sturdy, traditional VMs are "heavyweight". A standard VM boot involves emulating a legacy Basic Input Output System (BIOS), searching for a bootloader, and initializing a generic kernel with drivers for hardware that doesn't exist in the cloud, such as floppy drives. This led to the development of AWS Firecracker, a specialized VM monitor. Firecracker leverages the Linux Kernel-based Virtual Machine but strips away the bloat from the architecture.

The Role of the Kernel-based Virtual Machine is that AWS acts as a user-space (user side operations like a web browser) wrapper around it. It uses the ioctl(), or input output control, system call to communicate with the Linux kernel, which provides the hardware virtualization features.

Firecracker replaces complex hardware emulation with Virtual Input Output (VirtIO). It provides only four essential virtual devices: VirtIO-Net: For network packets, VirtIO-Block: For persistent storage access, Serial Console: For logging and debugging, and Partial Keyboard Controller: For system resets. By cutting the device model, Firecracker

eliminates the “probing” phase of a typical Operating System boot. The guest kernel is loaded directly into memory, allowing a Micro-Virtual Machine to reach a “Ready” state in around 125 milliseconds.

2.3 The Language Runtime Bottleneck

Once the Micro-Virtual Machine is active, the system enters the “Runtime INIT” phase. This is where the choice of programming language becomes a fundamental systems constraint.

In dynamic and interpreted runtimes, such as Python or NodeJs, they have a minimal “cold” footprint. The host just needs to load the interpreter binary into memory. Because these languages don’t require pre-compilation of the whole app before starting the event loop, they can begin executing logic within tens of milliseconds.

In contrast, managed runtimes utilizing Just-In-Time (JIT) compilation, like Java or C#, represent a high-latency tier within serverless architectures. The Java Virtual Machine, for example, must initialize a rigid memory management system, including the allocation of the Heap, before it can begin processing application code. This is followed by a heavy class-loading phase where the system must load and verify thousands of internal and third-party class files. The JIT compiler consumes significant CPU cycles during these early moments to analyze and optimize hot code paths, which often results in a massive CPU spike that delays the execution of the function handler. This architectural overhead means that a Java-based function requires a higher memory floor and a longer “warm-up” period to be ready compared to Python. This performance gap forces developers to pick between a fast start up with less efficiency with large amounts of data, like Python, and a slow start up but extreme efficiency once the app is running, like with Java.

2.4 The “Slim Start” Problem and the Cost of Libraries

A critical discovery in the SlimStart research paper highlights that the initialization phase is frequently a “memory-mapping” bottleneck rather than a computational one. Most modern applications rely on shared libraries to function. When a serverless function starts, the operating system loader must find these libraries, open them, and map their segments into the process’s virtual address space. This coordination between memory management and the file system is a complex task that adds a lot of overhead before the application code even runs.

The researchers behind SlimStart identified what is known as the “80% Rule”, finding that up to 80% of total startup time is spent solely on loading libraries. Much of this time is technically a waste, because a function might import a massive data science library like pandas or numpy, but only use a single utility function. The system still pays the full I/O cost

of reading the entire library, even if it doesn't use it. The system doesn't know which parts of the library will be used ahead of time, so it is forced to load the whole package.

This issue is further complicated by the way the CPU interacts with memory through page faults. As the runtime begins to access these newly mapped libraries, the CPU realizes the data isn't actually in physical RAM yet and triggers a fault. The operating system must then go and fetch the missing data from the virtual disk. In a cold start scenario, where the disk is often a network-attached volume located somewhere else in the data center, each page fault adds microseconds of latency. While a single fault is negligible, they aggregate by the thousands, turning into many seconds of delay that the user experiences as a slow "cold" response.

3 Mitigation Strategies

The technical limitations of the "cold start" have created a variety of mitigation strategies, ranging from simple resource over-engineering to completely redesigning how operating systems work. These methods are the backbone of cloud systems technology.

3.1 The Brute Force Method

The most straightforward approach to eliminating startup latency is Provisioned Concurrency, or "warm pools". In this model, the cloud provider maintains a specified number of execution environments that are fully initialized and kept in a "ready" state 24/7. When a request arrives, it is immediately routed to one of these warm workers, completely bypassing the download, loading and initialization phases. This method is fundamentally a "brute force" solution that negates the core serverless promise of "pay for what you use". From a systems efficiency perspective, this results in significant resource waste, as CPU and RAM are reserved even during periods of zero traffic. Also, since the August 2025 billing update, the cost of maintaining these idle environments has become a much larger consideration, forcing developers to balance the cost of "warmth" against the performance requirements of their users.

3.2 Balancing Start-up and Throughput

For managed runtimes like Java Virtual Machine, Tiered Compilation is a critical optimization used to bridge the gap between interpreted and compiled performance. In a standard environment, the Java Virtual Machine uses two JIT compilers: C1 is optimized for fast startup, and C2, which is optimized for peak long-term performance but takes longer to "warm up". In serverless environments, where functions are often short-lived, systems engineers often lock the runtime to C1-only compilation. By setting environment variables like "XX:TieredStopAtLevel=1", the runtime quickly produces optimized native code without

expensive analysis required by the C2 compiler. This reduces the initial CPU spike during the initialization phase, allowing the function to reach its execution handler much faster, but will be with a slightly slower peak throughput than a fully-warmed C2 optimized server.

3.3 The Evolution of SnapStart

One of the most advanced ways to fix cold starts is a technique called Checkpoint/Restore, which AWS uses in its SnapStart feature. Instead of starting a function from scratch every time it's called, the system initializes the function once during deployment and “freezes” its entire state, memory / disk, into an encrypted snapshot. When someone triggers the function, the system just unfreezes that snapshot inside a Micro-Virtual-Machine. This bypasses the slow boot-up process entirely, cutting delays for languages like Java and Python by up to 90% (taking a 5 second delay to 250ms).

While this “resurrection” of memory is fast, it creates two technical hurdles. First is the randomness problem: because the function is a perfect copy of a frozen state, it might use the same random numbers as other copies, which can break security and encryption. To fix this, the system has to insert new randomness into the function the moment it starts back up again. The second issue is of state connections: if a function was connected to a database when it was frozen, that connection will be broken when it wakes up later. To handle this, engineers use “runtime hooks” that allow the function to cleanly close its connections before it sleeps and restart them the instant it resumes.

4 Suggestions to the Field

While current technologies like AWS SnapStart and Firecracker have definitely reduced the “cold start tax”, the evolution toward a zero-latency cloud is an ongoing battle. Based on the analysis of the current system bottlenecks, several key areas for future research and engineering emerge.

4.1 Bypassing the Kernel with Unikernels

One of the most promising frontiers is the adoption of Unikernels. Currently, even lightweight Micro-VMs like Firecracker still run a Linux guest kernel. This kernel carries “ancestral” code, code that a single-purpose function does not need. Unikernels represent a shift where the application is compiled directly with only the specific operating system components it requires into a single, specialized machine image. By removing the traditional kernel/user-space boundary and the associated context switching, Unikernels could reduce boot times to single-digit milliseconds, making a cold start indistinguishable from a warm one.

4.2 Predictive Loading by Machine Learning

Currently, serverless scaling is reactive: a request comes, the system responds. A proactive approach using Predictive Loading could eliminate latency by anticipating traffic before it comes. By implementing Machine Learning models at the Control Plane level, providers could analyze historical traffic patterns to identify "burst" trends. If the system predicts a 90% probability of a traffic spike at 9:00AM, it can begin pre-warming instances at 8:59AM. This shifts the computational burden from the critical path of the user request to an idle background task, though it requires a delicate balance to avoid the financial waste of having the servers run overtime when not needed.

4.3 "Tree-Shaking" at the OS Level

Building on the "SlimStart" research, there is a clear need for more intelligent build tools. Currently, developers use "tree-shaking" to remove unused code, but the underlying OS libraries remain bloated. Future serverless platforms should offer OS-level tree-shaking, where the build process analyzes which system calls and shared libraries are actually needed. By stripping away unused segments in these libraries before the function is packaged, the memory-mapping bottleneck from section 2.4 could be significantly fixed, reducing the I/O overhead during the initialization phase.

5 Conclusion

The shift from physical servers to serverless computing has changed how we think about building and running applications. In this report, I looked at the "cold start" problem, which isn't just a small delay, but the result of a lot of different things happening at once behind the scenes, like how the system assigns resources, sets up virtual environments, and starts up different programming runtimes. What used to be mostly a performance issue has started to be a financial issue too, especially after cloud providers began charging for initialization time in 2025.

At the same time, things are improving fairly quickly. Tools like Firecracker and SnapStart have made a big difference by cutting down startup time and avoiding a lot of the usual setup work. Because of that, serverless systems are getting closer to feeling instantaneous. In the future, the gap between traditional servers and FaaS could get smaller. Cold starts may not entirely disappear, but they could be so small that users will never notice them.

References

AWS Lambda SnapStart configuration. (n.d.). Quarkus. Retrieved April 18, 2026, from https://quarkus.io/guides/aws-lambda-snapstart

AWS Lambda standardizes billing for INIT Phase. (2025, April 29). Amazon Web Services. https://aws.amazon.com/blogs/compute/aws-lambda-standardizes-billing-for-init-phase/

ByteBot. (2026, January 8). *Serverless cold start costs 2026: AWS INIT billing paradox*. Byteiota | From Bits to Bytes. https://byteiota.com/serverless-cold-start-costs-2026-aws-init-billing-paradox/

Firecracker. (n.d.). Retrieved April 18, 2026, from https://firecracker-microvm.github.io/

Firecracker – lightweight virtualization for serverless computing. (2018, November 26). Amazon Web Services. https://aws.amazon.com/blogs/aws/firecracker-lightweight-virtualization-for-serverless-computing/

Golec, M., Walia, G. K., Kumar, M., Cuadrado, F., Gill, S. S., & Uhlig, S. (2023, October 12). *Cold start latency in serverless computing: A systematic review, taxonomy, and future directions*. arXiv.Org. https://arxiv.org/abs/2310.08437

Lambda SnapStart runtime hooks for Python. (n.d.). AWS Lambda. Retrieved April 18, 2026, from https://docs.aws.amazon.com/lambda/latest/dg/snapstart-runtime-hooks-python.html

Mohammad Tariq, S. S., Al Zein, A., Vaidya, S. S., Khanolkar, A., Song, Z., & Roy, P. (2025). Efficient serverless cold start: Reducing library loading overhead by profile-guided optimization. *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*, 297–307. https://doi.org/10.1109/icdcs63083.2025.00037

Murugesh. (2026, February 27). AWS Lambda cold start: 7 proven fixes 2026. *AgileSoftLabs Blog*. https://www.agilesoftlabs.com/blog/2026/02/aws-lambda-cold-start-7-proven-fixes

Netflix case study. (n.d.). Amazon Web Services, Inc. Retrieved April 18, 2026, from https://aws.amazon.com/solutions/case-studies/netflix-case-study/

Scale to Zero AWS kit docs. (n.d.). Scale to Zero AWS Kit Docs. Retrieved April 18, 2026, from https://docs.scaletozeroaws.com/intro

Serverless cold start latency impact on API performance and user experience. (n.d.). PatSnap Eureka. Retrieved April 18, 2026, from https://eureka.patsnap.com/report-serverless-cold-start-latency-impact-on-api-performance-and-user-experience

Team, E. D. (2025, December 26). AWS lambda cold starts: why should you worry about them? *Edge Delta*. https://edgedelta.com/company/knowledge-center/aws-lambda-cold-start-cost

Under the hood: How AWS Lambda SnapStart optimizes function startup latency. (2025, August 19). Amazon Web Services. https://aws.amazon.com/blogs/compute/under-the-hood-how-aws-lambda-snapstart-optimizes-function-startup-latency/

Lakshmanan, R. (2022, May 26). *JVM c1, c2 compiler thread - High CPU consumption?* Fast Thread. https://blog.fastthread.io/jvm-c1-c2-compiler-thread-high-cpu-consumption/